

Estructura de ramas:

Rama	Propósito	Protección	Nace de	Se fusiona en
main	Código en producción. Solo se modifica mediante hotfix.	Bloqueada. No se permite push directo. Requiere PR y aprobación.	--(rama principal)	
develop	Integración continua. Aquí se unen todas las funcionalidades y correcciones.	Bloqueada. No se permite push directo. Requiere PR desde feature/... o bugfix/...	Se crea una vez desde el main	
feature/[area]/[desc]	Nuevas funcionalidades	Ninguna en el remoto (rama temporal). Se crea desde develop y se fusiona de vuelta con PR.	develop	develop
bugfix/[desc]	Corrección de errores reportados en develop .	Ninguna (rama temporal).	develop	develop
hotfix/[desc]	Parche urgente sobre producción.	Especial: Revisión de al menos 2 personas (área implicada + MW/QA). PR hacia main y develop .	main	main y develop

Ejemplos de nombres de ramas:

- **feature/backend/schema-base-de-datos**
- **feature/frontend/ventana-vista-profesores**
- **bugfix/horario-mal-generado**

Flujo de trabajo:

- **Nicole/Paola:** Crean nueva rama desde **develop** para trabajar en sus issues, hacen PR a **develop** y esperan a que Manuel y su líder de equipo aprueben el commit:
 - **Si se aprueba:** Se hace **Squash and Merge** a **develop**, se cierra la rama y se espera a un nuevo issue.
 - **Si no se aprueba:** Hacen los cambios marcados y vuelven a hacer PR.
- **Manuel:**
 - Hacer Test a los Pull Request de Nicole y Paola para verificar que no haya bugs, los test tienen que ser scripts de código que prueben distintas opciones o vulnerabilidades que puede tener el

código a revisar.

- Hacer despliegues de la aplicación desde la rama **develop** para verificar su estabilidad y si es cómoda de usar antes de hacer un PR a **main**
- Trabajar en la comunicación entre las funcionalidades del front y el back.

- **Daniel/Luis:**

- Asignación de los issues a sus respectivos equipos.
- Revisión de los PR hechos en sus áreas asignadas.
- Trabajar en funciones e implementaciones, ya sean de issues propios o ayudando en sus áreas.
- Los únicos con permiso de hacer los Squash and Merge a la rama main.

- **Todos:**

- Documentación del proyecto, tanto en código como en documentación, mas adelante se hablara sobre las convenciones para la documentación y los comentarios.

Requisitos para validar un Pull Request (PR):

- ☐ ¿Las variables están en español y siguen convenciones?
- ☐ ¿Las funciones complejas tienen su encabezado de documentación?
- ☐ ¿Si se cambió un flujo lógico, se actualizó el diagrama en Markdown/Mermaid?
- ☐ ¿Se deshabilitaron los **console.log** o **print** de depuración?
- ☐ ¿La funcionalidad esta bien implementada?

Notas:

issues: Tarea asignada en el tablero de github project (mas abajo se explica).

Pull Request(PR): Petición de revisión de código, se hace desde github despues de hacer un push de su rama en la que esta trabajando (**NO ES LA RAMA MAIN** Si se intenta hacer un push a la rama main del repo de github, la pagina se las va a rechazar por configuración).

Squash and Merge: Es combinar todos los commit de una rama en uno solo para posteriormente hacer un merge a otra rama (Esto lo trabajaran solo Manuel, Daniel y Luis).

Gestión de tareas y Convención de los commits.

Código de colores de las etiquetas (issues)

Etiqueta	Color	Uso
backend	azul	Tarea exclusiva del equipo de backend
frontend	verde	Tarea exclusiva del equipo de frontend
bug	rojo	Error encontrado en cualquier entorno
testing	amarillo	Relacionado con pruebas o QA

Etiqueta	Color	Uso
middleware	naranja	Implementar comunicación entre front y back
documentación	gris	Tarea relacionada con un cambio en la documentación del proyecto
enhancement	morado	Mejora opcional
refactor	negro	Refactorizar una funcionalidad
clean	blanco	Limpiar y formatear un código

Tablero del proyecto.

- **Backlog:** Tareas e ideas futuras.
- **To-Do:** Tareas por realizar.
- **In-Progress:** Tareas en progreso.
- **In-Review:** Pull Request a la espera de revisión u tarea de middleware.
- **Done:** Tareas terminadas.

Convención para la redacción de commits.

Tipos de commits:

- **feat:** Nueva funcionalidad.
- **fix:** Corrección de errores.
- **docs:** Cambio en la documentación.
- **style:** Cambio en el estilo de los comandos y el tabulado del código (no hay cambio lógico).
- **refactor:** reestructurar una sección del código
- **test:** añadir o modificar un test.

Alcances:

- backend
- frontend
- middleware
- qa
- core (afecta todos)

Estructura:

Título:"tipo(alcance): breve descripción"

Cuerpo:"descripción extendida" (Esto es opcional)

Fin-Cuerpo:"referencia a issue '#ID'"

- Ejemplo:

fix(backend): validar formato de email en login (#42)

feature(frontend): añadir formulario de ingreso de docente (#12)

Convención para la nomenclatura, documentación y comentado:

Nomenclatura en español.

Elemento	Formato	Ejemplo
Clases/Tipos	PascalCase	UsuarioAdmin, ValidadorMiddleware
Funciones/Métodos	camelCase	obtenerDatos(), validarToken()
Variables/Atributos	camelCase	nombreUsuario, esValido
Constantes	SCREAMING_SNAKE	LIMITE_INTENTOS, URL_API
Privados	_underscore	_instanciaPrivada

Reglas de oro:

- **Verbos para funciones:** `calcularTotal()` en lugar de `total()`.
- **Sustantivos para variables:** `listaTareas` en lugar de `hacerTareas`.
- **Booleanos con prefijo:** `esValido`, `tienePermiso`, `estaCargando`.

Comentarios y estructura interna.

Usaremos la sintaxis de comentarios de **Doxygen** para documentar el código y generar los archivos de una vez, se adjunta ejemplo de su sintaxis, esto es lo básico, pueden buscar más en caso de ser requerido.

```
/**
 * @brief Calcula el balance de carga entre los nodos del middleware.
 *
 * Esta función utiliza un algoritmo de round-robin para distribuir
 * las peticiones entrantes.
 *
 * @param nodos Lista de nodos activos disponibles.
 * @return int El ID del nodo seleccionado para la tarea.
 * @note Si la lista de nodos está vacía, devuelve -1.
 */
int calcularBalance(vector<int> nodos);
```

Etiquetas:

- **@brief:** Es el "título" o resumen rápido. Aparecerá en las listas generales de funciones del proyecto. Debe ser una sola línea.
- **Texto libre (Debajo de brief):** El párrafo que explica el algoritmo *round-robin* es la **descripción detallada**. Aquí es donde explicas la lógica pesada que el equipo (especialmente el de Middleware) necesita entender.

- **@param**: Define los argumentos que recibe la función.
 - Sintaxis: `@param [nombre_del_parámetro] [descripción]`.
 - En tu caso, documenta que `nodos` es el vector de entrada.
- **@return**: Explica qué devuelve la función y qué significa ese valor (en este caso, un `int` que representa un ID).
- **@note**: Se usa para resaltar advertencias, precondiciones o comportamientos importantes. Aquí es vital porque le avisa al programador qué pasa en un caso de error (lista vacía).

Comentarios de alerta:

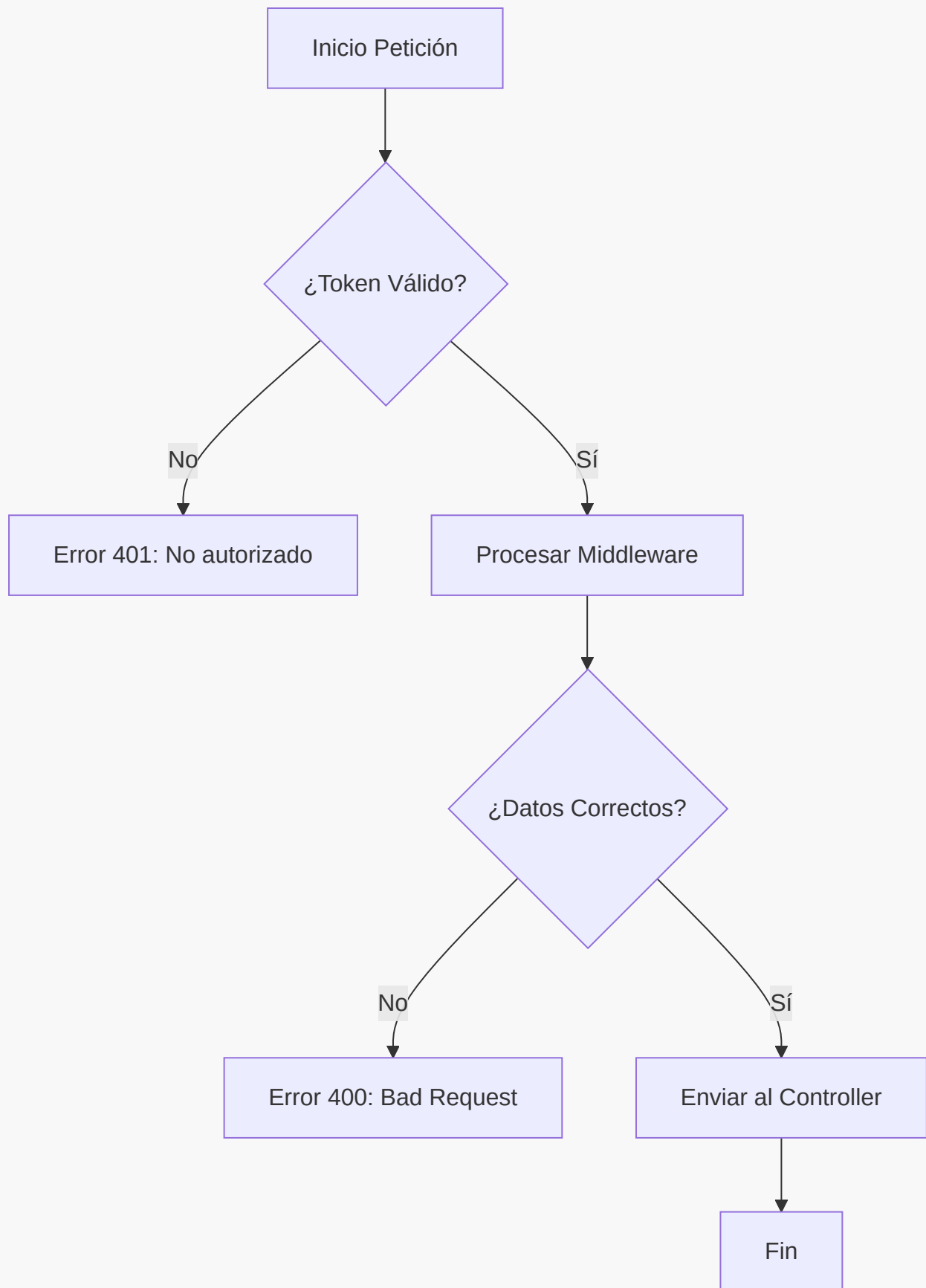
Esto es para que Manuel, Daniel, Luis sepan que ver y porque.

- `// TODO: Tareas pendientes`
- `// FIXME: Código que funciona pero está "roto" o es ineficiente`
- `// NOTE: Explicación de una decisión logica extaña o indecisa`

Diagramas de flujo:

Usaremos el lenguaje Mermaid.js para la creación de los diagramas de flujo, la razon es porque github, obsidian, y varios renders de archivos en formato .md (Markdown) soportan este lenguaje para renderizar automaticamente estos diagramas, se adjunta un ejemplo:

```
graph TD
  A[Inicio Petición] --> B{¿Token Válido?}
  B -- No --> C[Error 401: No autorizado]
  B -- Sí --> D[Procesar Middleware]
  D --> E{¿Datos Correctos?}
  E -- No --> F[Error 400: Bad Request]
  E -- Sí --> G[Enviar al Controller]
  G --> H[Fin]
```



Estructura de la documentación en el repo:

- **README.md**: Descripción general, cómo instalar y cómo ejecutar el proyecto.

- **CONTRIBUTING.md**: Aquí estarán las convenciones de la nomenclatura.
- **/docs**:
 - **arquitectura.md**: Diagramas de flujo de alto nivel y modelo de datos.
 - **api.md**: Endpoints (si hay back-end).
 - **qa_protocol.md**: Pasos que Manuel, Daniel, Luis debe seguir para aprobar un PR.
 - **test_protocol.md**: Pasos que Manuel debe seguir para generar los test de una funcionalidad.

Arquitectura del proyecto

```

gestor-horarios/
├── database/                                # Todo lo relacionado con la base de datos
│   ├── esquema.sql                        # Estructura si usaran SQLite (opcional al
inicio)                                    #
│   ├── seed_data.sql                     # Datos de prueba para desarrollo
│   └── migraciones/                      # Scripts de evolución futura
├── config/                                # Archivos de configuración de la aplicación
│   ├── app_config.json                   # Parámetros por defecto (ej. horarios
máximos, rutas)
│   ├── logging.ini                       # Configuración de logs
│   └── constraints/                      # Reglas de negocio por institución
│       └── robert_serra.json             # Restricciones específicas del liceo
├── resources/                             # Todo lo visual o estático que empaquetas
con la app
│   ├── icons/                           # Iconos del sistema
│   ├── styles/                          # Hojas de estilo Qt (qss)
│   ├── fonts/                           # Tipografías
│   └── qrc/                              # Archivos .qrc de Qt para compilar recursos
│       └── resources.qrc
├── docs/                                  # Documentación del proyecto
│   ├── arquitectura.md
│   ├── manual_usuario.md
│   └── api_interna.md                   # Contratos entre front/back (interfaces de
middleware)
│       └── test_protocol.md
├── src/                                   # CÓDIGO FUENTE - Separado en áreas
│   ├── backend/                          # Lógica de negocio y motor (Luis, Nicole)
│   │   └── core/                         # Entidades puras: Docente, Aula, Seccion,
Horario
│   │       ├── solver/                  # Algoritmo de generación con OR-Tools
│   │       ├── persistence/             # Acceso a datos: leer/escribir JSON/SQLite
│   │       └── CMakeLists.txt           # Target: libbackend (STATIC)
│   └── frontend/                         # Interfaz gráfica Qt (Daniel, Paola)
│       └── views/                       # Ventanas y diálogos

```

```

|   |   |   | widgets/           # Componentes reutilizables (ej:
tablaHorario)
|   |   |   |   | CMakeLists.txt # Target: libfrontend (STATIC, depende de
Qt::Widgets)
|   |   |   |   |
|   |   |   |   | middleware/    # Comunicación y adaptación (Manuel)
|   |   |   |   |   | controllers/ # Lógica de presentación/middleware
(ViewModels)
|   |   |   |   |   | adapters/    # Convierten tipos backend ↔ frontend
|   |   |   |   |   |   | validators/ # Reglas de validación que usa la UI
|   |   |   |   |   |   | CMakeLists.txt # Target: libmiddleware (STATIC, enlaza
frontend + backend)
|   |   |   |   |   |
|   |   |   |   |   | app/         # Punto de entrada y configuración global
|   |   |   |   |   |   | main.cpp # Arranca Qt, carga config, instancia
controladores
|   |   |   |   |   |   | CMakeLists.txt # Target ejecutable: GestorHorarios
|   |   |   |   |   |   |
|   |   |   |   |   |   | CMakeLists.txt # Agrupa subdirectorios (add_subdirectory)
|   |   |   |   |   |
|   |   |   |   |   | tests/
|   |   |   |   |   |   | unit/    # Pruebas de backend, sin UI (Nicole, Luis)
|   |   |   |   |   |   |   | backend/
|   |   |   |   |   |   |   | CMakeLists.txt
|   |   |   |   |   |   |   | integration/
|   |   |   |   |   |   |   | (Manuel)
|   |   |   |   |   |   |   | CMakeLists.txt
|   |   |   |   |   |   |   | ui/
|   |   |   |   |   |   |   | (opcional)
|   |   |   |   |   |   |   | CMakeLists.txt
|   |   |   |   |   |
|   |   |   |   |   | docker/
|   |   |   |   |   |   | Dockerfile.dev
|   |   |   |   |   |   | docker-compose.yml
|   |   |   |   |   |
|   |   |   |   |   | temp/
repo (gitignore) # Archivos temporales generados, fuera del
|   |   |   |   |   | .gitignore
|   |   |   |   |   | CMakeLists.txt # Raíz: find_package, opciones,
add_subdirectory
|   |   |   |   |   | README.md
|   |   |   |   |   | CONTRIBUTING.md

```